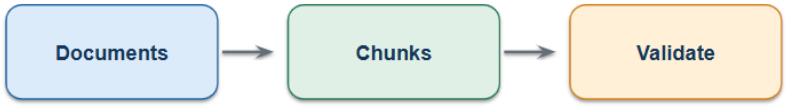
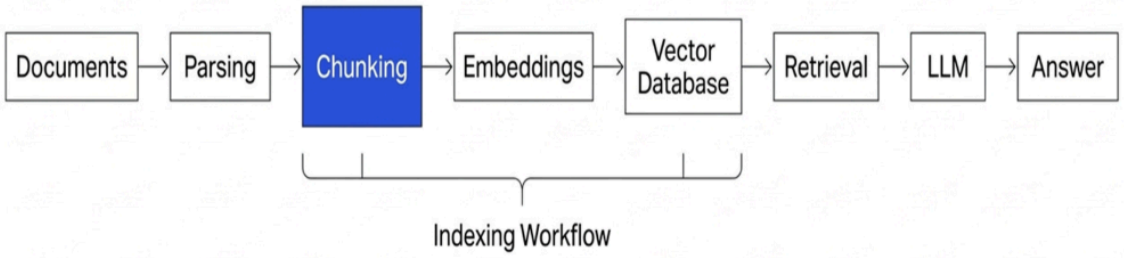
Lecture 7
Data Chunking Strategies for RAG-Based Chatbot Development



ITC-C508 – Deep Learning (NLP)
Prepared by: Dr. Rodolfo C. Raga Jr.

RAG begins with information retrieval—not generation

- A chatbot becomes more useful when an LLM can retrieve the right evidence before it answers.
- In RAG, the LLM is not asked to rely only on what it learned during training. Relevant external content is retrieved and supplied as context.



Chunking is the bridge between raw documents and searchable knowledge.

Why documents must be prepared before retrieval

RAG quality depends on whether the source document can be converted into clean, meaningful retrieval units.

1. **Context limits** — LLMs and embedding models process bounded amounts of text; feeding an entire manual is inefficient and often impossible.
2. **Retrieval precision** — A focused chunk gives the retriever a better chance of matching the user's question to the relevant passage.
3. **Computational efficiency** — Smaller units reduce unnecessary embedding, storage, retrieval, and prompt-processing work.
4. **Source fidelity** — Good preparation preserves headings, pages, sections, tables, and other metadata needed to explain where an answer came from.

THE CORE PROBLEM

A 200-page PDF is a document.

RAG needs retrievable evidence.

Preparation converts a document from a passive file into a structured knowledge source.

Think: extract → clean → structure → chunk → index

Chunking is an indexing decision

Chunking happens before retrieval: it determines the units that embeddings and vector search will later work with.

INDEXING / PREPARATION

1. **Parse —** extract text, headings, tables, page information, and other usable structure.
2. **Clean —** remove repeated headers, footers, artifacts, and irrelevant formatting.
3. **Chunk —** divide content into units that preserve enough meaning to stand alone.
4. **Embed —** convert each chunk into a vector representation of its semantic content.
5. **Store —** keep vectors plus chunk text and metadata in a searchable knowledge base.

Key implication: if the indexing stage creates poor chunks, retrieval quality is constrained before the LLM ever sees the question.

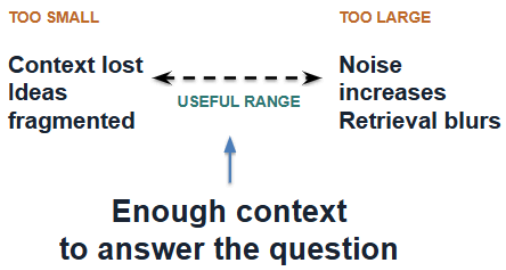
A chunk is a retrieval unit—not merely a piece of text

A good chunk is small enough to retrieve precisely but complete enough to preserve the idea needed to answer a question.

WHAT A CHUNK SHOULD ACHIEVE

1. **Self-contained meaning** — the chunk should make sense without requiring the entire document.
2. **Retrievable specificity** — it should contain enough distinctive information to match relevant questions.
3. **Context continuity** — important relationships should not be destroyed at chunk boundaries.
4. **Traceability** — the chunk should retain metadata that allows the system to identify its source.

THE CHUNK-SIZE TRADE-OFF



There is no universal best size. The right setting depends on document type, question type, model, and retrieval evaluation.

Five chunking strategies—and when to use them

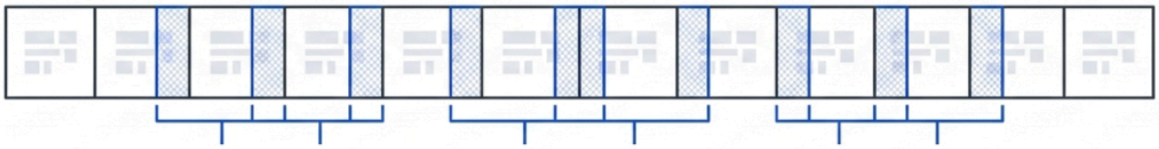
Chunking should follow the structure and information density of the source rather than a one-size-fits-all rule.

01 FIXED-SIZE	**Split by character, word, or token count.**	Simple baseline; easy to reproduce.
02 OVERLAPPING	**Repeat a controlled portion between adjacent chunks.**	Protects ideas that cross boundaries.
03 SENTENCE / PARAGRAPH	**Split at natural language boundaries.**	Preserves readable units and local meaning.
04 SEMANTIC	**Group text when adjacent content remains conceptually related.**	Useful when topics shift within a document.
05 STRUCTURE-AWARE	**Follow headings, sections, chapters, tables, or code structure.**	Strong fit for manuals, policies, reports, and technical documents.

Foundational Strategies: Fixed and Overlapping Methods



Fixed-Size Chunking: Segments text into equally sized pieces based on character or token counts. Fast and uniform, but ignores natural breaks. Best for simple logs.



Overlapping Chunking: Introduces a percentage buffer (e.g., 10-20%) between fixed cuts. Preserves immediate continuity of ideas across harsh boundaries.

Overlapping chunks preserve continuity—but add cost

Overlap is a deliberate trade-off: it reduces boundary loss while increasing duplicated text and index size.

WITHOUT OVERLAP	WITH OVERLAP
... policy applies to all units.	Chunk A: ... eligibility requires three
Eligibility requires three conditions.	Chunk B: eligibility requires three conditions.
The request must be submitted in advance.	Chunk B: ... submitted in advance.

WHY USE IT?

A question may depend on information located near a chunk boundary.

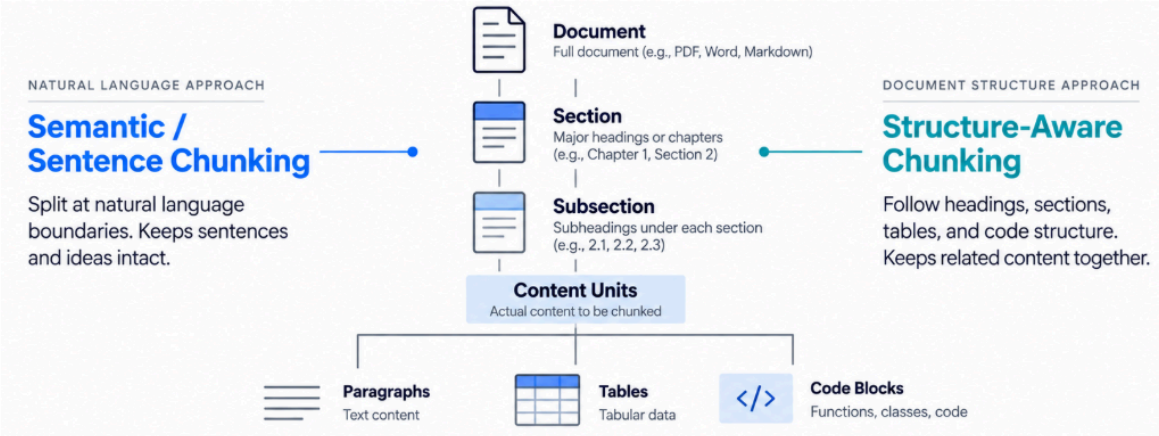
TRADE-OFF

More overlap means more repeated content, embeddings, storage, and possible retrieval redundancy.

STARTING POINT

For general prose, a modest overlap—often around 10–20%—is a reasonable experiment, not a universal rule.

Chunk by meaning and document structure – not arbitrary length



Result: Chunks are more coherent and easier to retrieve accurately

Choose chunking strategy by document structure

The best strategy changes with how information is organized and how users are likely to ask questions.

DOCUMENT TYPE	LIKELY STRUCTURE	STARTING STRATEGY	WHY
Policy / manual	chapters → sections → rules	Structure-aware	Keeps rules with their context
Research paper	abstract → sections → paragraphs	Paragraph + semantic	Preserves argument flow
FAQ / knowledge base	question → answer blocks	Paragraph / semantic	Each answer can stand alone
Code / technical docs	classes → functions → blocks	Structure-aware	Avoids breaking executable units
Scanned PDF	page images → OCR text	OCR + structure-aware	Text must be recovered before chunking

Principle: preserve the unit in which the source itself expresses meaning.

Chunking parameters should be treated as experimental settings

Start with sensible defaults, then evaluate retrieval rather than assuming a fixed chunk size is optimal.

CHUNK SIZE	OVERLAP	METADATA
~200–500 tokens	**~10–20%**	**source + page + section**
A useful starting range for general prose; larger values may suit dense technical passages.	Provides continuity at boundaries; increase only when evaluation shows boundary loss.	Enables filtering, traceability, citations, and better interpretation of retrieved content.

CONTEXT PRESERVATION — **Keep headings, section labels, neighboring context, table relationships, and page references when they are necessary to interpret the chunk correctly.**

Do not optimize chunking by size alone—optimize for retrieval usefulness.

Good chunking is a retrieval-quality problem

The final test is not whether chunks look neat—it is whether they help the system retrieve the right evidence and answer accurately.

1 BUILD	2 TEST	3 INSPECT	4 REFINE
Create a baseline chunking strategy.	**Ask representative questions.**	**Examine retrieved chunks.**	**Adjust size, overlap, or structure.**

LOOK FOR

- **Relevance** — did retrieval find the evidence needed?
- **Completeness** — is enough context present?
- **Noise** — are irrelevant passages being retrieved?

MEASURE

- **Retrieval quality** — precision / recall or retrieval hit rate.
- **Answer quality** — correctness, groundedness, and completeness.
- **Traceability** — can the answer be tied to the source?

KEY LESSON

- **Chunking is not a preprocessing detail to set once.**
- **It is a design variable that should be tested against the RAG task.**

A practical chunking workflow for PDF-based RAG

A reliable workflow separates document parsing, chunk creation, metadata, and retrieval evaluation.

01	**Inspect the PDF**	Text? tables? images? scanned pages?
02	**Extract & clean**	Recover text and remove repeated artifacts.
03	**Preserve structure**	Keep headings, sections, pages, tables.
04	**Choose strategy**	Start simple; match the document.
05	**Set parameters**	Size + overlap + metadata.
06	**Export chunks**	JSON/JSONL with source metadata.
07	**Evaluate retrieval**	Test with representative questions.

Recommended mindset: start with a transparent baseline, then improve only when retrieval evidence justifies the change.

The Golden Rules of Chunking

1 **There is no universal strategy.**
Match your segmentation method to your document's inherent structure and your query complexity.

2 **Balance size and semantics.**
Chunks must be large enough to preserve meaning, but small enough to guarantee retrieval precision.

3 **Iterate continuously.**
Start with a baseline, measure keyword coverage and coherence, and refine against real-world LLM responses.